

Functional Test Cases for Roman Numeral Converter

Author: Guillermo Solans Millá, alu190416
Course: Software Quality, Universitat San Jorge
Date: April 2026

Context

The application under test is a Roman numeral converter. The two functions being tested live in `RomanConverterService.cs`:

- `ToRoman(int number)` converts an integer (1-3999) to its canonical Roman numeral string. Throws `ArgumentOutOfRangeException` for numbers outside that range.
- `ToInteger(string roman)` converts a canonical Roman numeral string to an integer. Trims whitespace, uppercases the input, validates each character, computes the value using the subtractive rule, and performs a round-trip cross-validation to reject non-canonical forms. Throws `ArgumentException` for any invalid input.

SET 1: Manual Test Cases

1.1 Methodology

These test cases were designed **without AI assistance**, using two black-box techniques:

Domain Partition: the input space is divided into classes where every value behaves identically. One representative per class is sufficient to cover it.

Boundary Value Analysis (BVA): defects cluster at the edges between classes. For each boundary, three values are tested: just below, at, and just above.

Decision Tables were considered but ruled out: both functions take a single independent input, so there are no interacting conditions to combine.

For `ToInteger`, the Roman numeral specification produces additional sub-classes within the invalid partition: empty/null inputs, strings with illegal characters, and non-canonical forms (illegal repetitions, illegal subtractive pairs). Each is a distinct equivalence class with its own representative.

1.2 `ToRoman(int number)`

Equivalence Classes

ID	Partition	Input range	Expected
EP-R1	Valid	1 to 3999	Canonical Roman string

ID	Partition	Input range	Expected
EP-R2	Below minimum	$n < 1$	<code>ArgumentOutOfRangeException</code>
EP-R3	Above maximum	$n > 3999$	<code>ArgumentOutOfRangeException</code>

Test Cases

TC	Input	Expected output	Technique
TC-R-01	<code>ToRoman(-1)</code>	<code>ArgumentOutOfRangeException</code>	EP-R2, negative number
TC-R-02	<code>ToRoman(0)</code>	<code>ArgumentOutOfRangeException</code>	BVA, just below lower boundary
TC-R-03	<code>ToRoman(1)</code>	<code>"I"</code>	BVA, lower boundary
TC-R-04	<code>ToRoman(2)</code>	<code>"II"</code>	BVA, just above lower boundary
TC-R-05	<code>ToRoman(3999)</code>	<code>"MMMCMXCIX"</code>	BVA, upper boundary
TC-R-06	<code>ToRoman(4000)</code>	<code>ArgumentOutOfRangeException</code>	BVA, just above upper boundary
TC-R-07	<code>ToRoman(4)</code>	<code>"IV"</code>	EP-R1, subtractive pair (representative)
TC-R-08	<code>ToRoman(3)</code>	<code>"III"</code>	EP-R1, maximum legal repetition (1×3)

1.3 `ToInteger(string roman)`

Equivalence Classes

ID	Partition	Description	Expected
EP-I1	Valid canonical	Well-formed Roman numeral in [I, MMMCMXCIX]	Corresponding integer
EP-I2	Null / empty / whitespace	No parseable content	<code>ArgumentException</code>
EP-I3	Invalid characters	Contains chars outside {I,V,X,L,C,D,M}	<code>ArgumentException</code>
EP-I4	Non-canonical form	Valid chars but not canonical	<code>ArgumentException</code>

Test Cases

TC	Input	Expected output	Technique
TC-I-01	<code>ToInteger("I")</code>	<code>1</code>	BVA, lower boundary
TC-I-02	<code>ToInteger("MMMCMXCIX")</code>	<code>3999</code>	BVA, upper boundary

TC	Input	Expected output	Technique
TC-I-03	ToInteger("IV")	4	EP-I1, subtractive pair IV
TC-I-04	ToInteger("IX")	9	EP-I1, subtractive pair IX
TC-I-05	ToInteger("XL")	40	EP-I1, subtractive pair XL
TC-I-06	ToInteger("CM")	900	EP-I1, subtractive pair CM
TC-I-07	ToInteger("xlii")	42	EP-I1, lowercase input (normalisation)
TC-I-08	ToInteger(" XIV ")	14	EP-I1, leading/trailing spaces (normalisation)
TC-I-09	ToInteger("")	ArgumentException	EP-I2, empty string
TC-I-10	ToInteger(null)	ArgumentException	EP-I2, null
TC-I-11	ToInteger("ABC")	ArgumentException	EP-I3, invalid characters
TC-I-12	ToInteger("IIII")	ArgumentException	EP-I4, illegal repetition (I×4, should be IV)
TC-I-13	ToInteger("IC")	ArgumentException	EP-I4, illegal subtractive pair (I before C)
TC-I-14	ToInteger("IIX")	ArgumentException	EP-I4, two smaller symbols before a larger
TC-I-15	ToInteger("MCMXCIX")	1999	EP-I1, complex multi-symbol combination

Total Set 1: 8 + 12 = 20 test cases.

SET 2: AI-Assisted Test Cases

2.1 Prompts Used

The following prompts were submitted to an AI assistant in a single conversation.

Prompt 1: Context and initial cases

"I have two functions in a .NET 10 web API:

1. `ToRoman(int number)` converts an integer to a Roman numeral string. Throws `ArgumentOutOfRangeException` if outside [1, 3999].
2. `ToInteger(string roman)` converts a Roman numeral string to an integer. Trims whitespace, uppercases the input, validates characters, computes the value using the subtractive rule, and cross-validates by calling `ToRoman(result)` and comparing to the original input to reject non-canonical forms. Throws `ArgumentException` for any invalid input. Generate a set of functional black-box test cases for both functions grouped by category. For each test case provide: ID, input, expected output, and a brief rationale."

Prompt 2: Rule violations and edge cases

"Now add test cases specifically targeting Roman numeral rule violations (illegal repetitions, illegal subtractive pairs), and less evident edge cases such as the longest possible output, inputs with look-alike characters from other character sets, and the boundary where out-of-range and non-canonical overlap."

2.2 AI-Generated Test Cases

ToRoman(int number)

TC	Input	Expected output	Rationale
TC-AI-R-01	ToRoman(-1)	ArgumentOutOfRangeException	Negative input
TC-AI-R-02	ToRoman(0)	ArgumentOutOfRangeException	Just below minimum
TC-AI-R-03	ToRoman(1)	"I"	Minimum valid value
TC-AI-R-04	ToRoman(3999)	"MMMCMXCIX"	Maximum valid value
TC-AI-R-05	ToRoman(4000)	ArgumentOutOfRangeException	Just above maximum
TC-AI-R-06	ToRoman(9)	"IX"	Subtractive pair IX
TC-AI-R-07	ToRoman(3)	"III"	Maximum legal repetition
TC-AI-R-08	ToRoman(3888)	"MMMDCCCLXXXVIII"	Longest possible output (15 characters)

ToInteger(string roman)

TC	Input	Expected output	Rationale
TC-AI-I-01	ToInteger("I")	1	Minimum
TC-AI-I-02	ToInteger("MMMCMXCIX")	3999	Maximum
TC-AI-I-03	ToInteger("IV")	4	Subtractive pair IV
TC-AI-I-04	ToInteger("CM")	900	Subtractive pair CM
TC-AI-I-05	ToInteger("xlii")	42	Lowercase normalisation
TC-AI-I-06	ToInteger(" XLII ")	42	Whitespace trimming
TC-AI-I-07	ToInteger("")	ArgumentException	Empty string
TC-AI-I-08	ToInteger(null)	ArgumentException	Null input
TC-AI-I-09	ToInteger("IIII")	ArgumentException	I repeated 4 times
TC-AI-I-10	ToInteger("VX")	ArgumentException	V cannot appear in subtractive position
TC-AI-I-11	ToInteger("MMMM")	ArgumentException	M×4 equals 4000, which is out of range
TC-AI-I-12	ToInteger("IV̅")	ArgumentException	Greek Iota looks identical to I but is a different Unicode character

Total Set 2: 8 + 12 = 20 test cases.

Comparison and Analysis

3.1 Quantitative Overview

Metric	Set 1 (Manual)	Set 2 (AI-assisted)
ToRoman cases	8	8
ToInteger cases	12	12
Total	20	20

Metric	Set 1 (Manual)	Set 2 (AI-assisted)
Technique	Domain Partition + BVA	Prompt-driven, category-based

3.2 Key Differences

Methodology. Set 1 follows a formal two-step process: define equivalence classes first, then derive boundary values. Each test case is labelled with its class ID (EP-R1, EP-I4, etc.), which makes it easy to verify that every partition has at least one representative. Set 2 organises tests by natural-language categories, which is readable but harder to audit for completeness.

Boundary coverage. Set 1 applies the BVA rule of three points per boundary (below, at, above) and includes TC-R-04 (`ToRoman(2)`) as the explicit boundary+1 case. Set 2 covers the boundaries but does not distinguish them from general valid cases.

Domain rule coverage. Set 1 covers three distinct non-canonical sub-classes: illegal repetition (`IIII`), illegal subtractive pair (`IC`), and two smaller symbols before a larger (`IIX`). Set 2 covers only two of these, finding a different illegal subtractive case (`VX`) but missing the `IIX` pattern entirely.

Less evident edge cases. Set 2 found two cases Set 1 missed: `ToRoman(3888)` as the longest possible Roman output (15 characters), and `ToInteger("IV")` using a Greek Iota, a look-alike Unicode character that passes visual inspection but should be rejected. Neither of these follows from Domain Partition or BVA; they came from the AI drawing on broader knowledge.

Subtractive pair coverage. Set 1 covers four of the six valid subtractive pairs individually (IV, IX, XL, CM), while Set 2 covers only two (IV, CM). Set 1 is more thorough here; Set 2 traded that coverage for the edge cases above.

3.3 Conclusions

Both sets are similar in size and overlap significantly, but they differ in how they were built and where they end up. The manual set is more systematic and auditable; the AI-assisted set is faster and catches inputs that are harder to think of from first principles. Together they cover more ground than either does alone.